

```

import kagglehub
import os
import time
import glob
from os import listdir
from os.path import isfile, join
import matplotlib.pyplot as plt
import matplotlib.image as img
import pandas as pd
import re
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix, ConfusionMatrix
from IPython.display import Image

```

Data Citation Gupta, A., & Gupta, R. (2019). ALL Challenge dataset of ISBI 2019 [Data set]. The Cancer Imaging Archive. <https://doi.org/10.7937/tcia.2019.dc64i46r>

Kaggle Data Location: https://www.kaggle.com/datasets/andrewmvd/leukemia-classification?select=C-NMC_Leukemia

Train Data Prep

```

path = kagglehub.dataset_download("andrewmvd/leukemia-classification")

print("Path to dataset files:", path)

files = (glob.glob(f"{path}\\C-NMC_Leukemia\\*"))

d = {}

for i in range(len(files)):
    file = int(i)
    print(glob.glob(f"{files[file]}\\*"))
    d[i] = glob.glob(f"{files[file]}\\*")

tr = d[1]

tra = {}

for i in range(len(tr)):
    file = int(i)

```

```

print(glob.glob(f"{tr[file]}\\*"))
tra[i] = glob.glob(f"{tr[file]}\\*")

train_cancer = []

for i in range(len(tra)):
    train_cancer.extend(glob.glob(f"{tra[i][0]}\\*"))

len(train_cancer)

train_healthy = []

for i in range(len(tra)):
    train_healthy.extend(glob.glob(f"{tra[i][1]}\\*"))

len(train_healthy)

```

EDA of training data

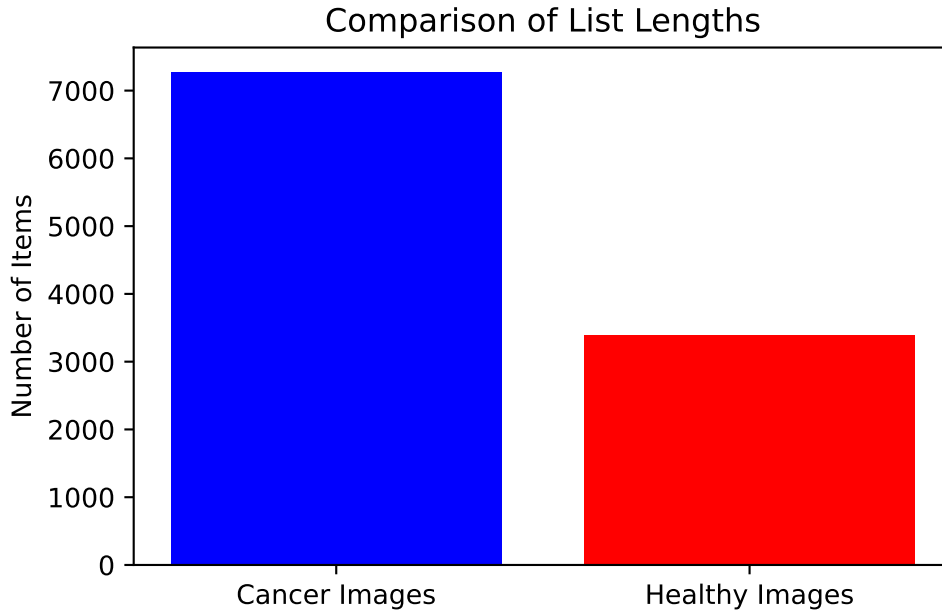
```

cancer_length = len(train_cancer)
healthy_length = len(train_healthy)

x = [0, 1]
lengths = [cancer_length, healthy_length]
colors = ['blue', 'red']

plt.bar(x, lengths, color=colors)
plt.xticks(x, ['Cancer Images', 'Healthy Images'])
plt.ylabel('Number of Items')
plt.title('Comparison of List Lengths')
plt.show()

```



Interesting to see a data set with so many more cancer images than healthy images!

In the realworld this is the opposite. I suspect many of the cancer patient images don't actually show Leukemia, but due to the child having Leukemia, all images are classified as such.

Also absolutely heartbreaking to think about so many Children with Leukemia.

Below we can see a quick breakdown of each file name (copied directly from the competition site):

“ “ ”

Cancer cell images' naming convention: `UID_P_N_C_all` `UID_P` -> where $P=1,2,\dots$ signifies the subject ID. `UID_P_N`: where $N=1,2,3,\dots$ represent the image number `UID_P_N_C`: where $C=1,2,3,\dots$ represents the cell count. (More than one cell can be found in a particular microscopic image) `UID_P_N_C_all`: The 'all' tag represent the class to which the cell belongs, in this case, 'ALL' or cancer class.

Similarly, the naming convention for normal (healthy) cell images is as follows: `UID_HS_N_C_hem`, where H denotes healthy/normal subject, S denotes the healthy subject's ID, N denotes the image number, C denotes the cell count, and hem tag, in the end, denotes the normal subjects' cell.

“ “ ”

One thing to mention as well which originally confused me, the data is segmented prior to this, so when we have the same image then have mutiple number of cells, that means prior to this the image was segmented to be one cell per image. Another thing, the `_all` means the patient

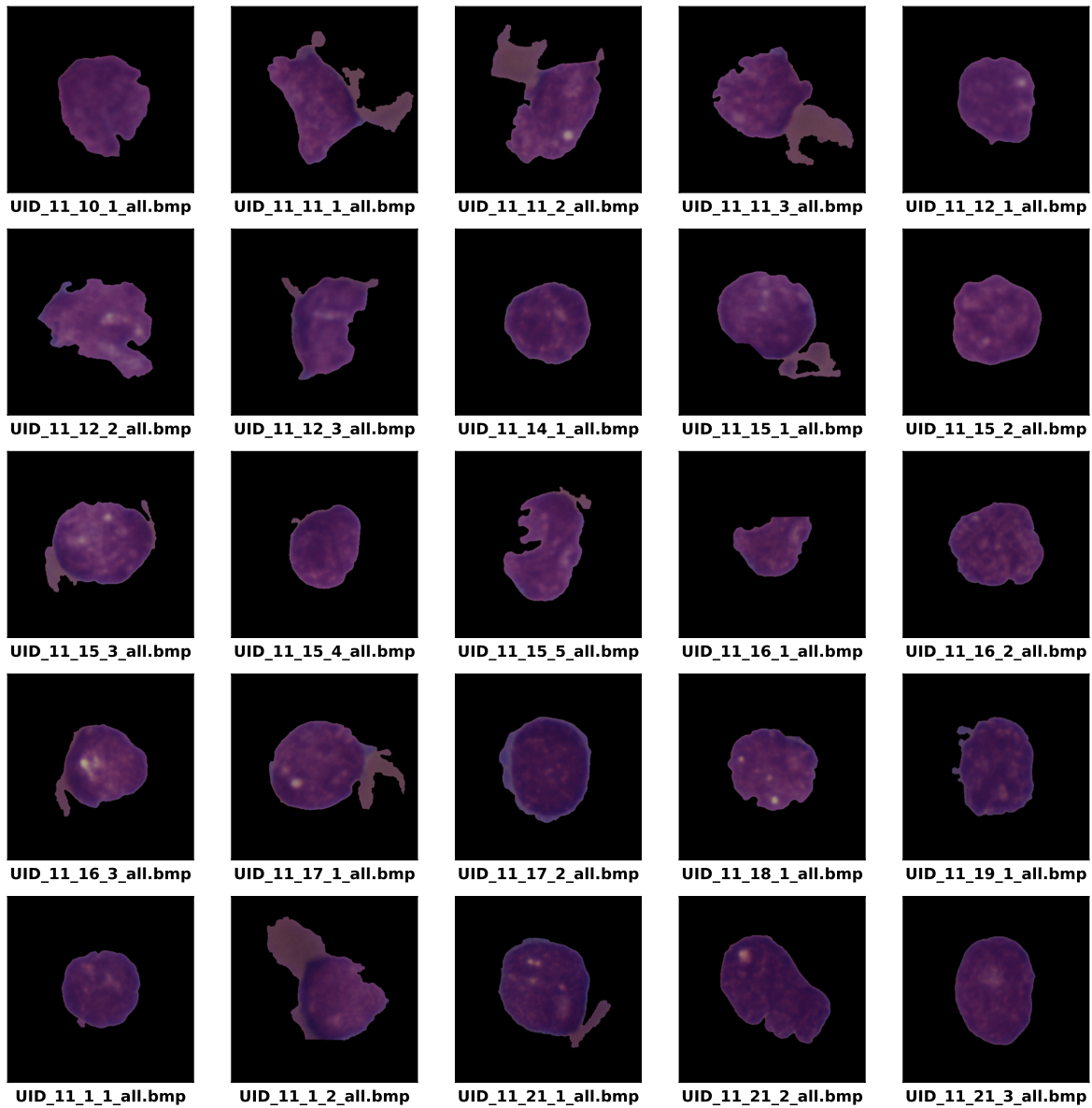
has Leukemia, while `_hem` means the patient does not have Leukemia. This also confused me as I assumed `_all` meant all patients, it does not.

I will also graph some of the images so you can get an idea of what we are working with!

```
fig = plt.figure(figsize=(10,10))
fig.suptitle('Cancer', fontsize=26, fontweight='bold')

for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    images = img.imread(train_cancer[i])
    plt.imshow(images)
    plt.xlabel(os.path.basename(train_cancer[i]), fontsize=8, fontweight = "bold")
plt.show()
```

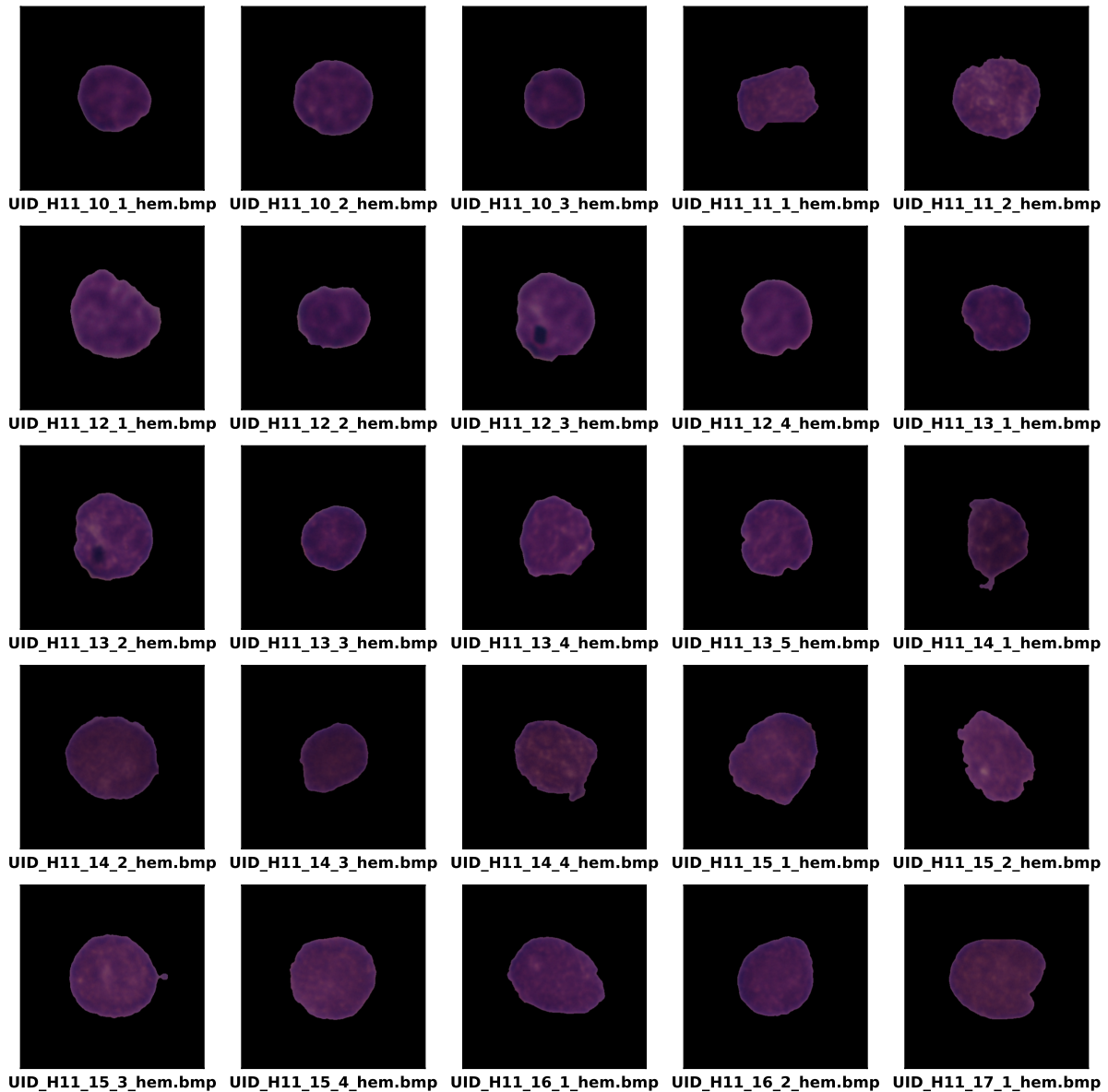
Cancer



```
fig = plt.figure(figsize=(10,10))  
fig.suptitle('Healthy', fontsize=26, fontweight='bold')  
  
for i in range(25):
```

```
plt.subplot(5,5,i+1)
plt.xticks([])
plt.yticks([])
plt.grid(False)
images = img.imread(train_healthy[i])
plt.imshow(images)
plt.xlabel(os.path.basename(train_healthy[i]), fontsize=8, fontweight = "bold")
plt.show()
```

Healthy



The cells definitely look different to me!

This data set is unique as there are only 118 individuals (before the private test competition set), some with up to hundreds of cell images. I do not think we need hundreds per person, as just by visually looking I can see a difference in many of the cells.

Now I can use some regex to make a new dataframe that adds the data from the file path. I also have to change the IDs, as the same ID is reused between the two data sets!

```
df_health = pd.DataFrame(train_healthy)

df_health = df_health.rename(columns = {0: "FilePath"})

df_health["ID"] = df_health["FilePath"].str.extract(r"([^\s]+)_hem.bmp$")

df_health["Patient"] = df_health["ID"].str.extract(r"UID_[Hh](\d+)_").astype(int)

# Working :D

df_cancer = pd.DataFrame(train_cancer)

df_cancer = df_cancer.rename(columns = {0: "FilePath"})

df_cancer["ID"] = df_cancer["FilePath"].str.extract(r"([^\s]+)_all.bmp$")

df_cancer["Patient"] = df_cancer["ID"].str.extract(r"UID_(\d+)_").astype(int)
```

EDA IDs

```
h1 = pd.DataFrame(df_health["Patient"].value_counts().sort_index())
h2 = pd.DataFrame(df_cancer["Patient"].value_counts().sort_index())

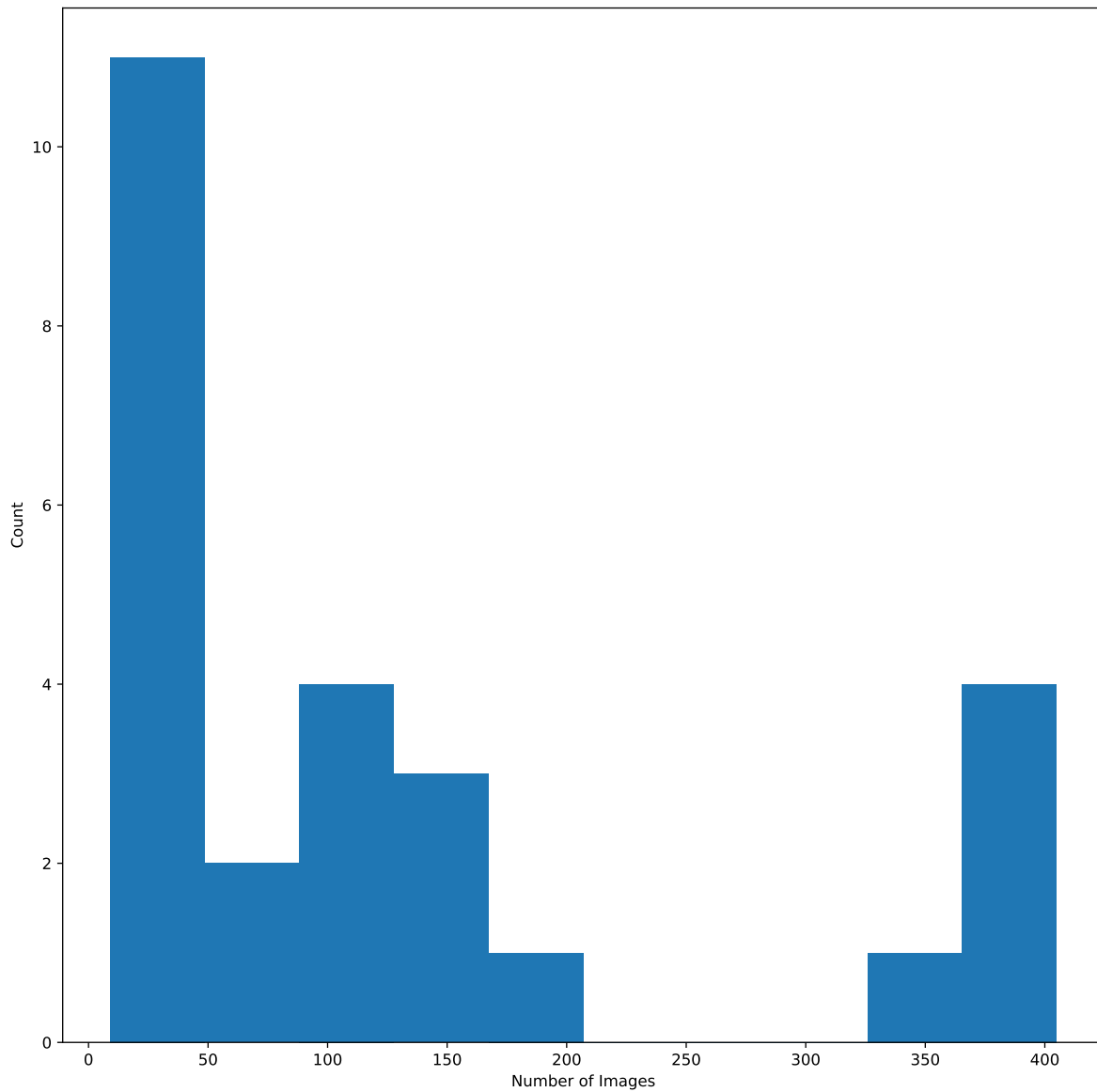
h1 = h1.reset_index()
h2 = h2.reset_index()
```

It seems that ID numbers are reused between healthy and non-healthy. We will have to fix this. We can also see a huge range between images for each ID. Single digit images may make this difficult!

We will look at two histograms to quickly see the spread between each for images.

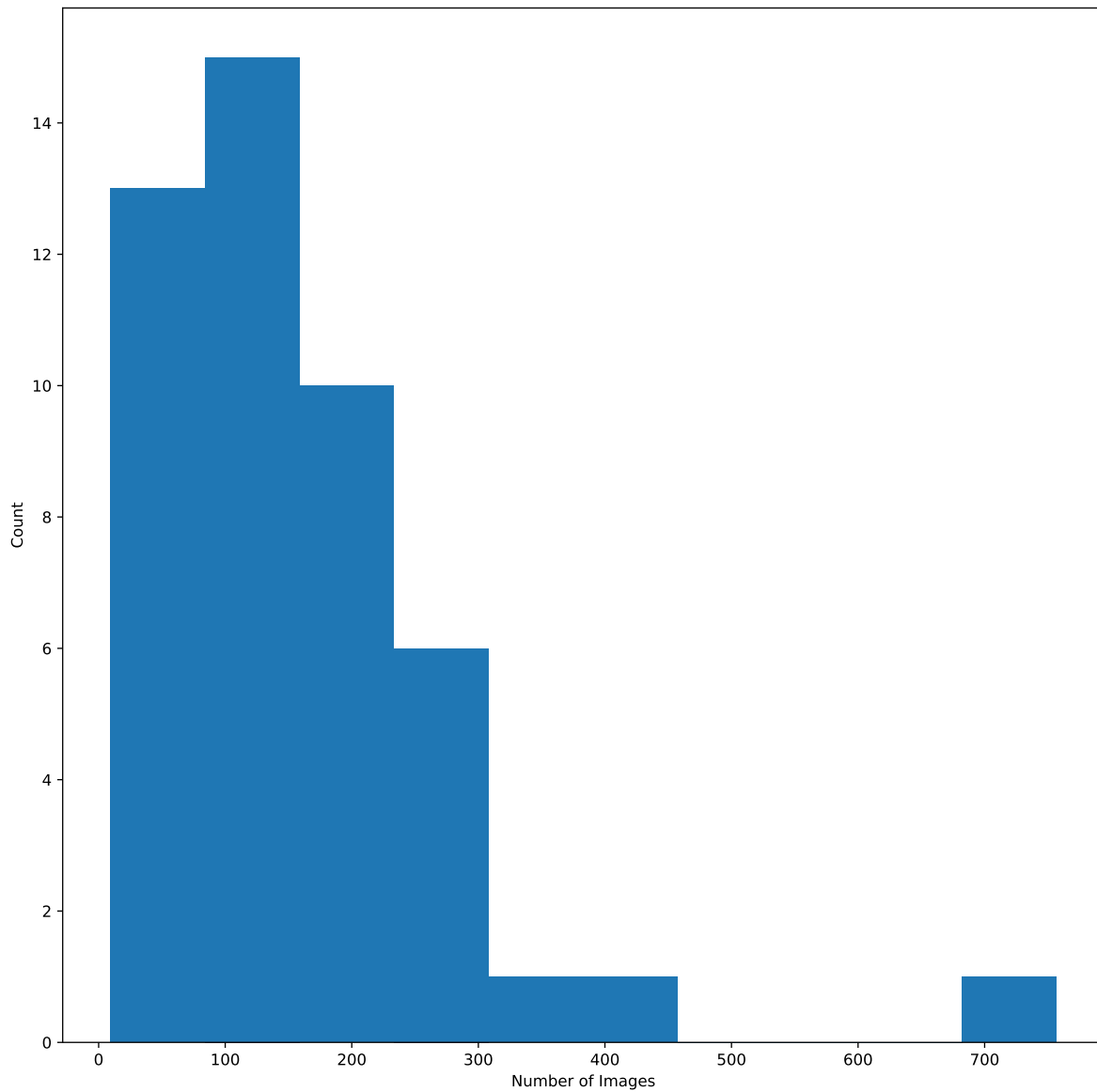
```
plt.figure(figsize = (12,12))
plt.hist(h1["count"])
plt.title("No Leukemia", fontsize=26, fontweight='bold')
plt.ylabel("Count")
plt.xlabel("Number of Images")
plt.show()
```


No Leukemia



```
plt.figure(figsize = (12,12))
plt.hist(h2["count"])
plt.title("Leukemia", fontsize=26, fontweight='bold')
plt.ylabel("Count")
plt.xlabel("Number of Images")
plt.show()
```

Leukemia



IDs to break up

```
h1_breakup = h1[h1["count"] > 200]
h1listbreak = list(h1_breakup["Patient"])
h2_breakup = h2[h2["count"] > 200]
```

```
h2listbreak = list(h2_breakup["Patient"])
```

Loop to break groups with a minimum of 100 in each group that have more than 200 images. This is used way later in the demo, and I had other ideas for it that didn't work out. Feel free to kinda skip over this it can be a headache to follow.

```
for i in h1listbreak:
    count = h1.loc[h1["Patient"] == i, "count"].values[0]
    max_index = max(df_health[df_health["Patient"] == i].index)
    min_index = min(df_health[df_health["Patient"] == i].index)
    breaks = (count // 100)
    jumps = (max_index - min_index) / breaks
    for b in range(1, breaks + 1):
        # if b == 1:
        #     continue
        if max_index - (min_index + (jumps * b)) < 100:
            break
        indexes_to_change_min = int(min_index + (jumps * b))
        indexes_to_change_list = list(range(indexes_to_change_min, max_index))
        changed_index = df_health.index.isin(indexes_to_change_list)
        df_health.loc[changed_index, "Patient"] = f"{i}_{b}"
```

```
for i in h2listbreak:
    count = h2.loc[h2["Patient"] == i, "count"].values[0]
    max_index = max(df_cancer[df_cancer["Patient"] == i].index)
    min_index = min(df_cancer[df_cancer["Patient"] == i].index)
    breaks = (count // 100)
    jumps = (max_index - min_index) / breaks
    for b in range(1, breaks + 1):
        # if b == 1:
        #     continue
        if max_index - (min_index + (jumps * b)) < 100:
            break
        indexes_to_change_min = int(min_index + (jumps * b))
        indexes_to_change_list = list(range(indexes_to_change_min, max_index))
        changed_index = df_cancer.index.isin(indexes_to_change_list)
        df_cancer.loc[changed_index, "Patient"] = f"{i}_{b}"
```

Resetting the IDs

```

df_health["Cancer"] = 0
df_cancer["Cancer"] = 1

full_df = pd.concat([df_health, df_cancer], axis = 0)

full_df["Patient_New_ID"] = pd.factorize(full_df["Patient"])[0] + 1

len(list(full_df["Patient"].unique()))

full_df.sort_values("Patient_New_ID")

df_combined = full_df[["FilePath", "Cancer", "Patient_New_ID"]]

df_combined["Cancer"] = df_combined["Cancer"].astype("str")

print(df_combined["Cancer"].value_counts())

```

```

Cancer
1      7272
0      3389
Name: count, dtype: int64

```

Here we can load in the Validation Data.

```

v = d[2]

df_val = pd.read_csv(v[1])

validation = []

validation.extend(glob.glob(f"{v[0]}\\*"))

df_val_combine = pd.DataFrame(validation, columns = ["FilePath"])

df_val_combine["ID"] = df_val_combine["FilePath"].str.extract(r"([^\]+[.bmp])$")

df_val = pd.merge(df_val_combine, df_val, left_on = "ID", right_on = "new_names")

df_val = df_val.drop(columns = ["new_names"])

df_val = df_val.rename(columns = {"labels" : "Cancer"})

```

```
df_val["Cancer"] = df_val["Cancer"].astype("str")

df_val["Clean_Patient_ID"] = df_val["Patient_ID"]
```

Here we create and save the Convolutional Neural Network. I originally played around with pretrained models, but they all did much worse! Like .1 AUC worse which is a huge gap. There is a large problem with overfitting, and the data can be difficult to train on

We use data augmentation in order to help stop overfitting. It is a difficult problem to predict (this way) as many “Cancer” calls don’t actually show cancer, but are from an individual with Leukemia. Not every cell is going to be visibly messed up, so the model is fed healthy cells and told they are cancerous!

Also the model weights are randomized. Our best validation AUC was slightly above .7, and your outcome may slightly vary. In a real world setting we probably wouldn’t want this randomization and I instead would share my best model!

```
datagen_train = ImageDataGenerator(
    rescale=1./255,
    horizontal_flip=True,
    vertical_flip=True,
    rotation_range=20,
    zoom_range=0.1
)

datagen_val = ImageDataGenerator(
    rescale=1./255
)

train_gen = datagen_train.flow_from_dataframe(
    dataframe=df_combined,
    x_col="FilePath",
    y_col="Cancer",
    target_size=(64, 64),
    batch_size=32,
    class_mode="binary",
    shuffle=True
)

val_gen = datagen_val.flow_from_dataframe(
    dataframe=df_val,
    x_col="FilePath",
```

```

y_col="Cancer",
target_size=(64, 64),
batch_size=32,
class_mode="binary",
shuffle=False
)

```

Here we have 2 hidden layers, in a pretty standard CNN. It uses batchnormalization, has 32 filters and a 6x6 size, then 64 filters with a 4x4 size. It uses a pretty high drop out of .5 to help with overfitting. It also uses Batchnormalization and Maxpooling (2,2) on both hidden layers. This is to make the model run faster as I am running it on an 8 gb laptop. It also uses a leakyrelu activation function which allows for slightly negative values to be passed through, helping with the dying relu problem.

```

model = models.Sequential([
    layers.Conv2D(32, (6,6), input_shape=(64, 64, 3)),
    layers.LeakyReLU(alpha=0.1),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.5),

    layers.Conv2D(64, (4,4)),
    layers.LeakyReLU(alpha=0.1),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.5),

    layers.Flatten(),
    layers.Dense(128),
    layers.LeakyReLU(alpha=0.1),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

model.summary()

```

We evaluate on AUC, and slowly decrease the learning rate as the model is better fit. It has an early stopping of 10, with 50 epochs, a learning rate of .0001, which is halved every 3 epochs with no improvement.

```

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='binary_crossentropy',
    metrics=[tf.keras.metrics.AUC(name='auc')]
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_auc',
    patience=10,
    mode='max',
    restore_best_weights=True
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_auc',
    factor=0.5,
    patience=3,
    mode='max',
    min_lr=0.000001
)

history = model.fit(
    train_gen,
    epochs=50,
    steps_per_epoch=100,
    validation_data=val_gen,
    callbacks=[early_stopping, reduce_lr]
)

# model.save_weights("Model_Basic.h5")

```

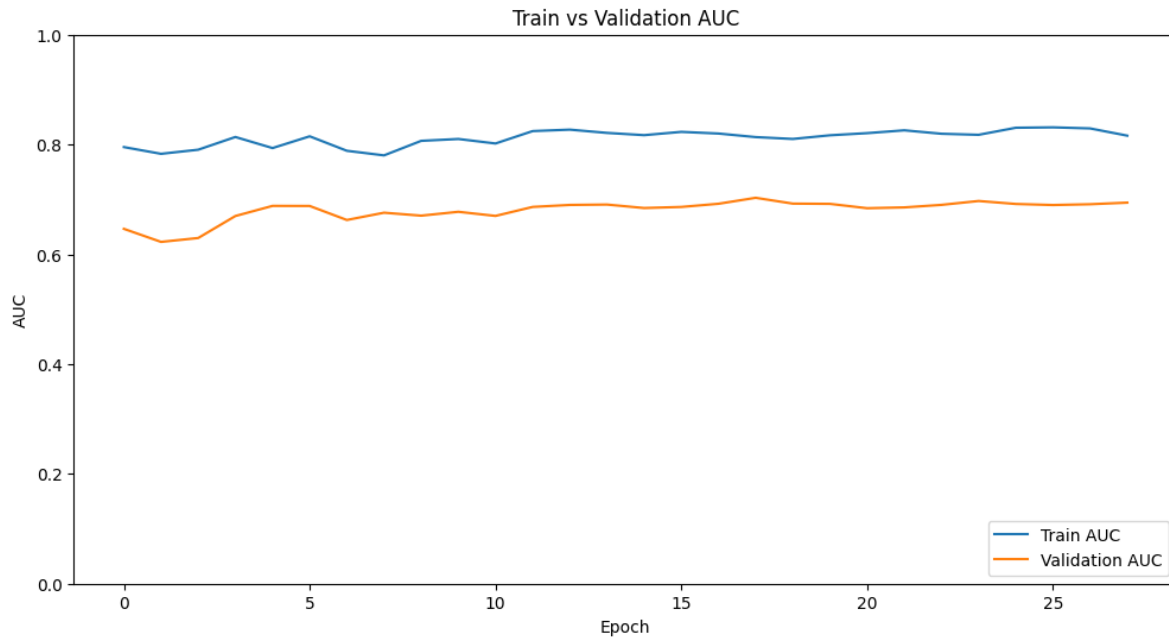
Here we plot the AUC to see if the model is overfitting or the validation AUC is tracking.

```

plt.figure(figsize=(12, 6))
plt.plot(history.history['auc'], label='Train AUC')
plt.plot(history.history['val_auc'], label='Validation AUC')
plt.xlabel('Epoch')
plt.ylabel('AUC')
plt.title('Train vs Validation AUC')
plt.legend(loc='lower right')
plt.ylim([0, 1])
plt.show()

```

```
Image("./AUC Graph.png")
```



Here we get the confusion matrix, which I love to look at! It always makes it way clearer what is going on for me.

```
# model = tf.keras.models.load_model('./Model_Basic.keras')
# df_val = pd.read_csv("./Validation_Data.csv")
# df_val["Cancer"] = df_val["Cancer"].astype("str")

pred_gen = datagen_val.flow_from_dataframe(
    dataframe=df_val,
    x_col="FilePath",
    y_col="Cancer",
    target_size=(64, 64),
    batch_size=32,
    class_mode="binary",
    shuffle=False
)

df_model_pred = df_val

df_model_pred["Predicted_Prob"] = model.predict(pred_gen)
```



```

df_model_pred["Predicted_Cancer"] = (df_model_pred["Predicted_Prob"] > 0.5).astype(int)
df_model_pred["True_Label"] = df_model_pred["Cancer"].astype(int)

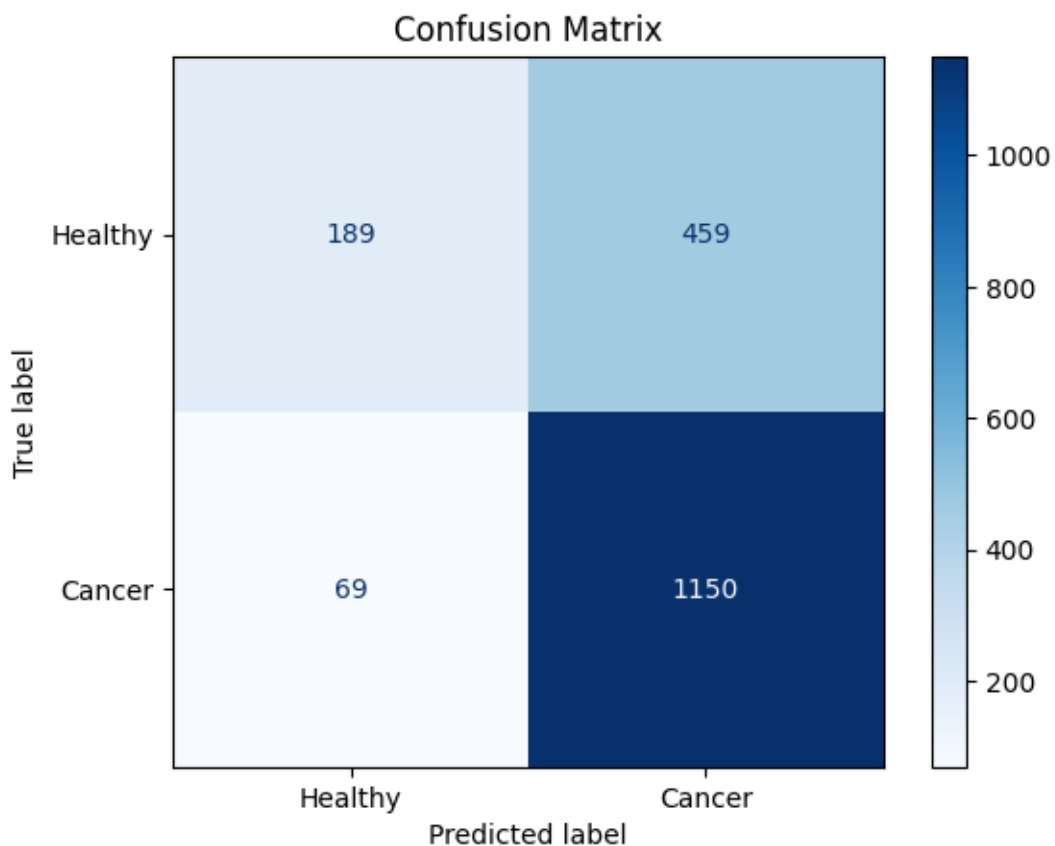
acc = accuracy_score(df_model_pred["True_Label"], df_model_pred["Predicted_Cancer"])
auc = roc_auc_score(df_model_pred["True_Label"], df_model_pred["Predicted_Prob"])

print(f"Accuracy: {acc:.4f}")
print(f"AUC:      {auc:.4f}")

cm = confusion_matrix(df_model_pred["True_Label"], df_model_pred["Predicted_Cancer"])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Healthy", "Cancer"])
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

Image("./ConfusionMatrix.png")

```



Feel free to ignore this, this is for the demos later!

```
df_combined.to_csv("All_Train_Data.csv", index = False, header = True)
df_val.to_csv("Validation_Data.csv", index = False, header = True)
model.save('Model_Basic.keras')
```